

Relatorio Tecnico — Mini E-commerce Distribuido

Atividade 1 — Sistemas Distribuidos

Lucas Calabria | Sistemas Distribuidos | 2025

1. Comunicacao entre Microservicos

A comunicacao entre os microservicos foi implementada via HTTP/REST sincrono. O API Gateway (porta 5000) atua como ponto de entrada unico e repassa as requisicoes aos servicos internos usando o cliente HTTP assincrono **httpx**. O Servico de Pedidos tambem realiza chamadas REST diretas ao Servico de Produtos para validar a existencia e o estoque do item antes de confirmar o pedido. Nao foi utilizado nenhum sistema de filas (como RabbitMQ ou Kafka), mantendo a simplicidade exigida pela atividade.

2. Estrategia de Consistencia na Replicacao

Foi adotada **consistencia forte** (*strong consistency*) para a replicacao do Servico de Produtos. O fluxo de escrita segue o protocolo *write-all-before-confirm*: toda operacao de escrita e primeiro propagada para a replica secundaria (porta 5012) via chamada HTTP ao endpoint interno **/internal/products**, e somente apos confirmacao de ambas as instancias o sucesso e retornado ao cliente. As leituras sao distribuidas entre primario e replica por *round-robin* simples no Gateway. A justificativa para consistencia forte e que dados de catalogo de produtos (precos e estoque) exigem visao uniforme para evitar vendas inconsistentes.

3. Tolerancia a Falhas — Queda do Servico de Pedidos

O sistema foi projetado para **degradacao graciosa** (*graceful degradation*). Se o Servico de Pedidos cair, o API Gateway detecta a falha apos 2 ciclos consecutivos de heartbeat (aproximadamente 10 segundos) e registra um log com timestamp. A partir desse momento, qualquer requisicao roteada para **/orders** retorna **503 Service Unavailable** com mensagem clara ao cliente. Os demais servicos (Usuarios e Produtos) continuam operando normalmente de forma independente, pois nao ha dependencia entre eles. Quando o Servico de Pedidos retornar, o Gateway registra a recuperacao em log automaticamente e volta a rotear requisicoes para ele.

4. JWT e Controle de Acesso por Papel (Role)

O Servico de Usuarios gera um JWT assinado com **HMAC-SHA256** contendo os campos *userId*, *email*, *role* (user ou admin) e *exp* (expiracao em 1 hora). Para criar produtos, o endpoint **POST /products** exige que o token contenha *role: admin*; caso contrario, o servico retorna **403 Forbidden**. O Gateway repassa o header *Authorization* intacto para os servicos internos, que validam o token de forma independente. Senhas sao armazenadas com hash SHA-256, nunca em texto plano. Assim, um usuario comum que tente criar produtos recebera 403 mesmo com token valido, pois seu campo role sera "user".

5. Limitacoes em Relacao a um Sistema Real de Producao

- Armazenamento em arquivos JSON nao suporta transacoes ACID nem acesso concorrente seguro; em producao usariamos PostgreSQL/MongoDB com locking.
- A replicacao e sincrona e bloqueante: se a replica estiver lenta, toda escrita e penalizada. Em producao, padroes como Raft ou replicacao assincrona com WAL seriam mais adequados.

- Não há *circuit breaker* (ex: Hystrix/Resilience4j) para evitar cascata de falhas.
- O JWT usa chave secreta simetrica compartilhada entre servicos; em producao seria preferivel assimetria (RS256) com rotacao de chaves.
- Não há *rate limiting*, TLS entre servicos internos nem observabilidade centralizada (metricas, tracing distribuido).
- O hash SHA-256 para senhas e mais vulneravel a ataques de forca bruta em comparacao com bcrypt/Argon2 com salt, que seriam a escolha correta em producao.